Instalación de una aplicación en Docker mediante docker-compose.yml

- Instalación de Docker
  - Introducción a Docker
  - Instalación de Docker Desktop
  - Ver logs de un contenedor
- Mover contenedores a otro equipo
- Automatizar instalaciones con archivos de docker-compose.yml
  - Ejemplos de docker-compose.yml
    - Nextcloud Server con MariaDB (para más de 4 o 5 usuarios concurrentes)
    - Nextcloud Server con SQLite (más ligera)
    - Moodle (con archivo con variables de entorno .env)

Descargar estos apuntes

Instalación de Docker

Introducción a Docker

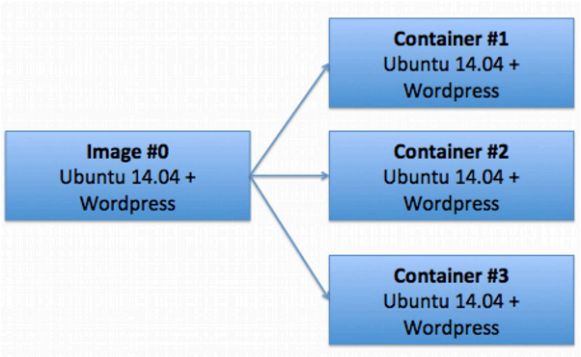
Docker es un proyecto de código abierto que automatiza el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción y automatización de virtualización de aplicaciones en múltiples sistemas operativos.

Docker es una **plataforma de virtualización** a nivel de sistema operativo que permite crear una aplicación y empaquetarla junto con sus dependencias y librerías en un contenedor que será capaz de ejecutarse en cualquier otra máquina que disponga de una capa para la gestión de dichos contenedores.

Una **imagen** es una especificación **estática** de lo que debería ser el contenedor en tiempo de ejecución, incluido el **código de la aplicación** y las **configuraciones de tiempo de ejecución**.

Un **contenedor** es la **instanciación de una imagen**, y puede haber múltiples instancias de un mismo contenedor.

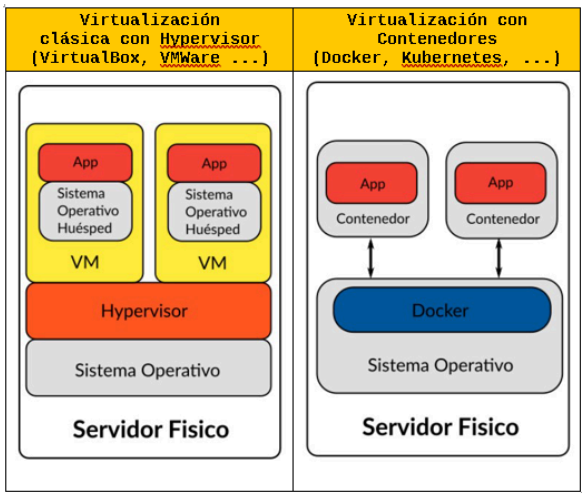
Por ejemplo, a partir de una imagen que contenga un Ubuntu + Wordpress podríamos crear 3 contenedores ejecutándose de manera independiente:



Hay dos versiones de Docker, una libre y otra comercial. Nosotros trabajaremos con **Docker Community Edition**, que es la versión libre.

La virtualización clásica permite que sus sistemas operativos (Windows o Linux) se ejecuten simultáneamente en un solo sistema de hardware.

En la virtualización con contenedores las aplicaciones instaladas **comparten el mismo kernel del sistema operativo** y separan los procesos de las aplicaciones del resto del sistema.



Los **contenedores** permiten desplegar aplicaciones **más rápido**, arrancarlas y pararlas más rápido y **aprovechar mejor los recursos de hardware**.

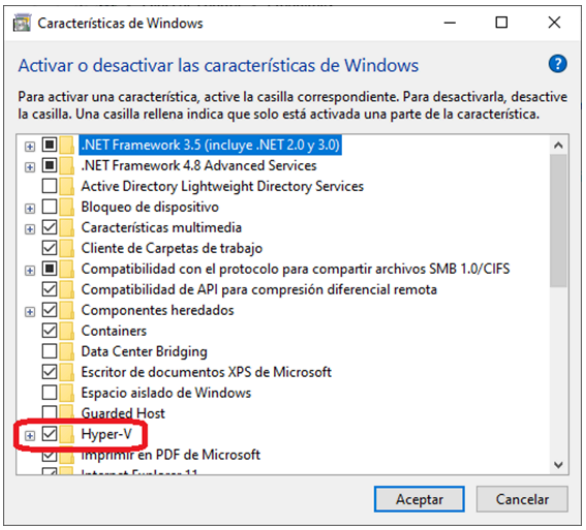
Las máquinas virtuales nos permiten crear sistemas completos totalmente aislados, con mayor control sobre el entorno y mezclando sistemas operativos host y huésped.

Instalación de Docker Desktop

Para instalar Docker utilizaremos, descargándolo desde aquí: Docker Desktop

Ejecutar el instalador y seguir los pasos. Al finalizar la instalación, es posible que se nos pedirá reiniciar el sistema.

La instalación de Docker activa el Hyper-V de Windows en "activar y desactivar características de Windows", lo que provoca en ocasiones algún problema si también tenemos VirtualBox.



Docker se ejecuta en segundo plano y muestra un icono en la barra de herramientas.

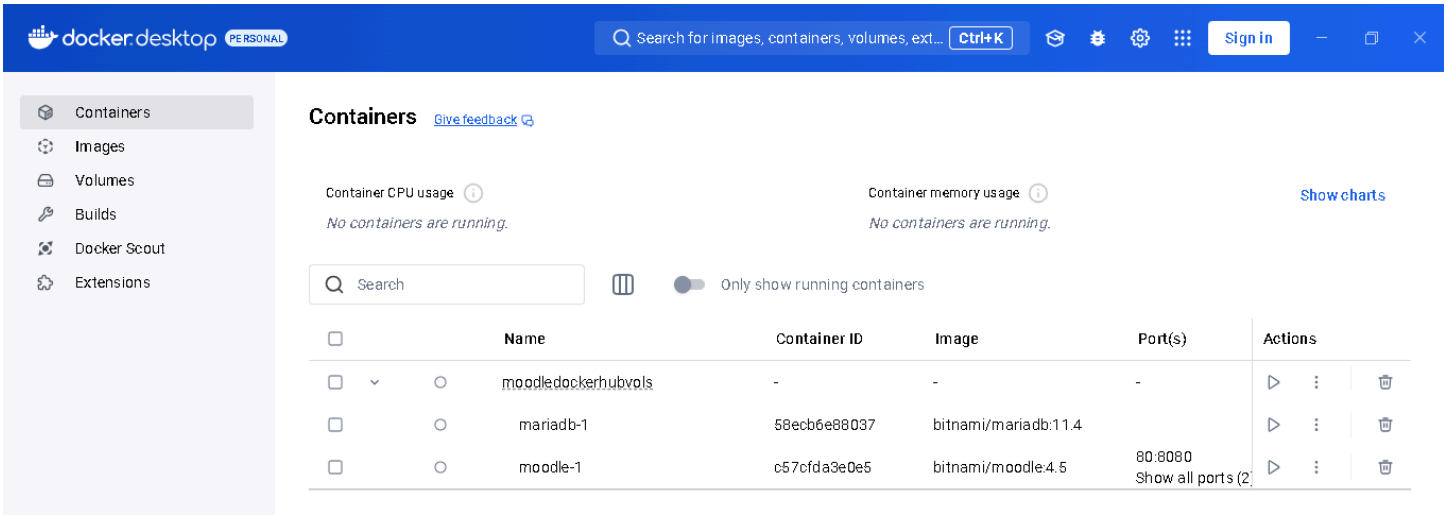
Al abrir el programa veremos que tenemos Contenedores, Imágenes, Volúmenes y Redes.

En **contenedores** veremos los contenedores que tenemos creados y si están en marcha o no.

En **imágenes** veremos las imágenes que tenemos descargadas en nuestro repositorio local.

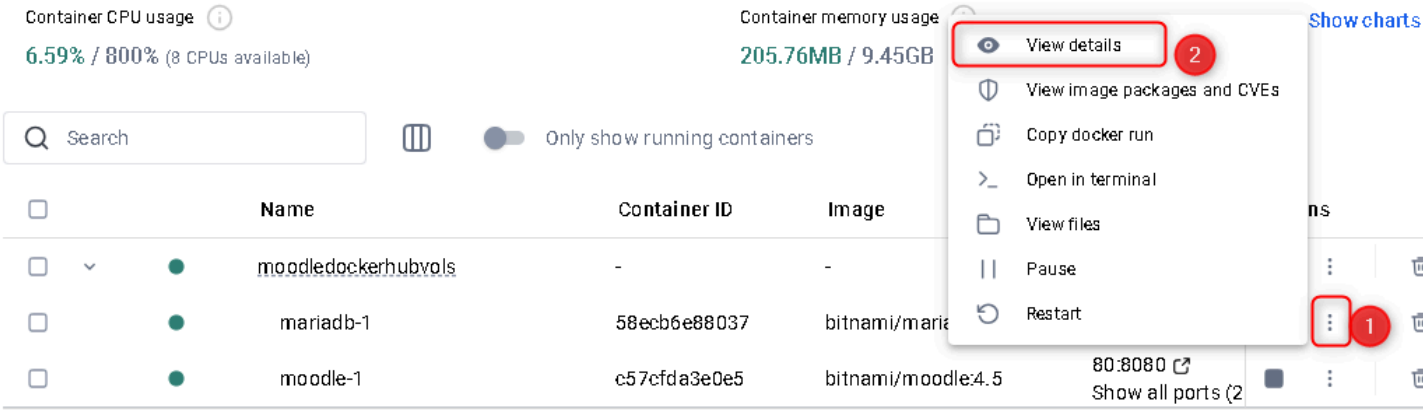
En **volúmenes** veremos los volúmenes que tenemos creados para guardar datos de los contenedores.

> ⚠ **Vólumnes de datos**
>
> Si queremos llevarnos los datos de la aplicación a otro equipo, debemos copiar la carpeta donde se guardan los volúmenes de datos. En Windows suele ser `C:\Users\Usuario\AppData\Local\Docker\wsl\data\ext4.vhdx`. Esta carpeta contiene todos los volúmenes de datos de Docker, y si la borramos perderemos todos los datos de las aplicaciones que tengamos instaladas en Docker.
> La mejor opción es **crear los volúmenes de datos dentro de la carpeta del proyecto**, para que podamos copiarlos a otro equipo sin problemas. Esto se hace en el archivo **docker-compose.yml**, indicando la ruta donde queremos guardar los datos con "." delante.



Ver logs de un contenedor

Si el contendedor no está en marcha al darle al play, podemos ver los logs desde los 3 puntos de la derecha del contenedor.



Veremos en el log que error ha habido si no ha arrancado correctamente.



Mover contenedores a otro equipo

Si tenemos una carpeta con un proyecto de Docker, podemos moverlo a otro equipo copiando la carpeta completa. En el otro equipo, abrimos Docker Desktop y desde la terminal nos movemos a la carpeta del proyecto y ejecutamos:

```
docker-compose up -d
```

Esto será más rápido que volver a descargar las imágenes, ya que las imágenes se descargan en el primer equipo y se copian a la carpeta del proyecto. En el segundo equipo, Docker Desktop detectará que ya tiene las imágenes necesarias y no las volverá a descargar.

Puede darte un error de permisos del directorio de datos:

> "Su directorio de datos puede ser leído por otros usuarios. Por favor, cambie los permisos a 0770"

Nextcloud comprueba que nextcloud/html/data no sea legible por "otros" (permisos tipo 777/755). Te ha pasado al copiar y pegar la carpeta porque Windows (o el método de copia) le ha puesto permisos demasiado abiertos al restaurarla.

Para quitar esta restricción de Nextcloud, abrimos el archivo nextcloud\html\config\config.php y añadimos esta línea dentro del array (por ejemplo, justo después de la línea 'datadirectory' => ...):

```
'check_data_directory_permissions' => false,
```

> **Para mayor agilidad en el aula os proporcionare la carpeta del proyecto con la aplicación ya instalada y funcionando, para que solo tengáis que copiarla a vuestro equipo y arrancar los contenedores con docker compose up -d.**

Automatizar instalaciones con archivos de docker-compose.yml

En la mayoría de instalaciones necesitamos varios contenedores para ofrecer servicios en diferentes contenedores pero dependientes entre sí. Por ejemplo, tener un servidor apache con PHP y otro servidor con mysql.

En estos casos, es mucho mejor utilizar **docker compose**. Esta utilidad dispone de un script con extensión **yml** que configura estos servicios.

Por lo tanto, para instalar cualquier aplicación web con docker realizaremos los siguientes pasos:

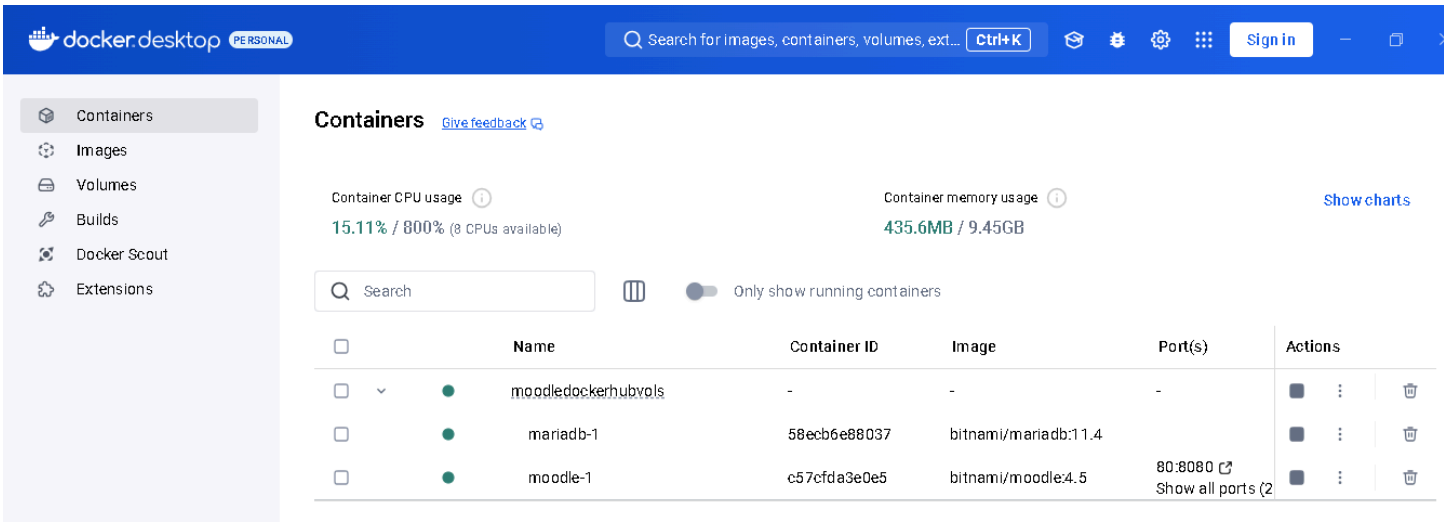
1. **Crear una carpeta** para el proyecto
2. **Copiar en la carpeta** el correspondiente archivo **docker-compose.yml** según la aplicación que queramos instalar. Es posible que necesitemos crear alguna carpeta adicional para guardar los datos persistentes de la aplicación (bases de datos, archivos subidos, etc). Estas carpetas deben estar indicadas en el archivo docker-compose.yml. También puede ser necesario crear un archivo **.env** para definir variables de entorno.
3. Desde el terminal estando ubicados en la carpeta del proyecto donde esta ubicado docker-compose.yml, **ejecutar** el siguiente comando:

   ```
   docker-compose up -d
   ```

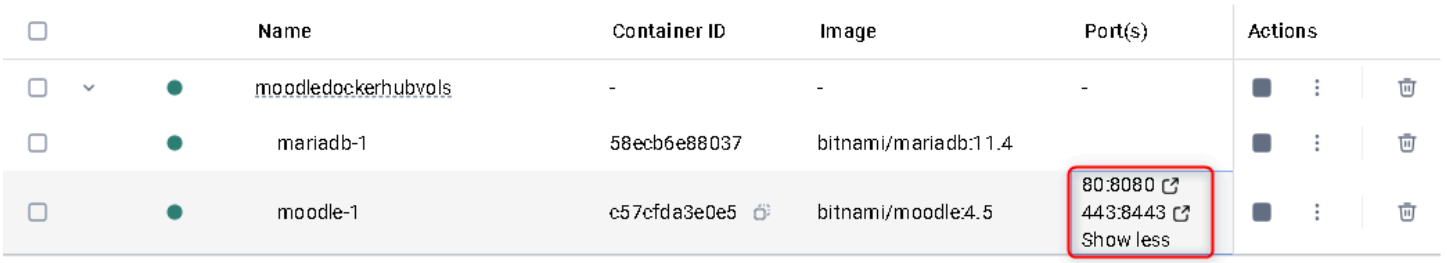
   Si el archivo se llama de otra forma, deberemos indicar el nombre del archivo con el parámetro -f, por ejemplo:

   ```
   docker-compose -f archivo.yml up -d
   ```

4. **Comprobar** en Docker que todo ha ido bien y está el servicio en marcha.



Para **acceder a la aplicación** podemos ver desde aquí la URL con el puerto que está utilizando. Normalmente será localhost:puerto.



> **En el primer arranque, el contenedor de Nextcloud copia todo el código (~1 GB descomprimido) desde la imagen a ./nextcloud/html porque has montado esa carpeta. En un PC lento del aula esto puede tardar varios minutos. Señal de que va bien: la carpeta ./nextcloud/html va llenándose de archivos y docker compose logs nextcloud no muestra errores. No es raro que en ese intervalo el navegador no cargue nada.**

Ejemplos de docker-compose.yml

Nextcloud Server con MariaDB (para más de 4 o 5 usuarios concurrentes)

```
# docker-compose.yml — Nextcloud PORTABLE con MariaDB
# Para llevarlo a otro equipo: copiar ESTA carpeta entera y ejecutar: docker compose up -d
services:
  nextcloud:
    image: nextcloud:29.0.16-apache   # versión EXACTA de la copia maestra (no hacer pull nunca)
    restart: unless-stopped
    depends_on:
      - db
    ports:
      - "8900:80"
    volumes:
      - ./nextcloud/html:/var/www/html  # código y datos de archivos
    environment:
      - MYSQL_PASSWORD=nextcloud
      - MYSQL_DATABASE=nextclouddb
      - MYSQL_USER=nextcloud
      - MYSQL_HOST=db
      - PHP_MEMORY_LIMIT=512M
      - OVERWRITEPROTOCOL=http
      - OVERWRITECLIURL=http://localhost:8900
    mem_limit: 512m

  db:
    image: mariadb:11.8               # versión EXACTA de la copia maestra (no hacer pull nunca)
    restart: unless-stopped
    command: >-
      --transaction-isolation=READ-COMMITTED
      --binlog-format=ROW
      --innodb-buffer-pool-size=64M
      --skip-name-resolve
      --performance-schema=OFF
    volumes:
      - ./db:/var/lib/mysql             # BD EN CARPETA: imprescindible para la portabilidad
    environment:
      - MYSQL_ROOT_PASSWORD=nextcloud
      - MYSQL_PASSWORD=nextcloud
      - MYSQL_DATABASE=nextclouddb
      - MYSQL_USER=nextcloud
    mem_limit: 256m
```

Nextcloud Server con SQLite (más ligera)

```yaml
# docker-compose.yml — Nextcloud PORTABLE con SQLite
# Para llevarlo a otro equipo: copiar ESTA carpeta entera y ejecutar: docker compose up -d
services:
  nextcloud:
    image: nextcloud:29.0.16-apache   # versión EXACTA de la copia maestra (no hacer pull nunca)
    restart: unless-stopped
    ports:
      - "8900:80"
    volumes:
      - ./nextcloud/html:/var/www/html  # TODO viaja aquí: código, datos y BD SQLite
    environment:
      # Solo actúan si la carpeta viene vacía (instalación automática):
      - NEXTCLOUD_ADMIN_USER=profesor
      - NEXTCLOUD_ADMIN_PASSWORD=CambiarEstaClave.2026
      - SQLITE_DATABASE=nextcloud
      - PHP_MEMORY_LIMIT=512M
      - OVERWRITEPROTOCOL=http
      - OVERWRITECLIURL=http://localhost:8900
    mem_limit: 512m
```

Moodle (con archivo con variables de entorno .env)

```yaml
services:
  mariadb:
    image: mariadb:10.11
    container_name: moodle_mariadb
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: ${MARIADB_ROOT_PASSWORD}
      MARIADB_DATABASE: ${MARIADB_DATABASE}
      MARIADB_USER: ${MARIADB_USER}
      MARIADB_PASSWORD: ${MARIADB_PASSWORD}
    volumes:
      # Base de datos persistente dentro del proyecto
      - ./data/db:/var/lib/mysql
    #command: >
    #  --innodb_buffer_pool_size=256M
    #  --innodb_log_file_size=64M
    #  --innodb_flush_log_at_trx_commit=2
    #  --skip-name-resolve

  moodle:
    #image: bitnamilegacy/moodle:latest
    image: erseco/alpine-moodle:v5.1.0
    container_name: moodle_app
    restart: unless-stopped
    depends_on:
      - mariadb
    ports:
      - "8080:8080"
    environment:
      MOODLE_SITE_URL: ${MOODLE_SITE_URL}

      MOODLE_DATABASE_TYPE: mariadb
      MOODLE_DATABASE_HOST: mariadb
      MOODLE_DATABASE_PORT_NUMBER: 3306
      MOODLE_DATABASE_NAME: ${MARIADB_DATABASE}
      MOODLE_DATABASE_USER: ${MARIADB_USER}
      MOODLE_DATABASE_PASSWORD: ${MARIADB_PASSWORD}

      MOODLE_USERNAME: ${MOODLE_ADMIN_USER}
      MOODLE_PASSWORD: ${MOODLE_ADMIN_PASS}
      MOODLE_EMAIL: ${MOODLE_ADMIN_EMAIL}

      # Ajustes conservadores (equipos lentos)
      #PHP_MEMORY_LIMIT: 384M
      #PHP_MAX_EXECUTION_TIME: 300
      #PHP_POST_MAX_SIZE: 64M
      #PHP_UPLOAD_MAX_FILESIZE: 64M
```

```
    volumes:
      # Moodle y moodledata dentro del proyecto
      - ./data/moodle:/bitnami/moodle
      - ./data/moodledata:/bitnami/moodledata
```

En un archivo **.env** podemos definir las variables de entorno:

```
MOODLE_SITE_URL=http://localhost:8080

MARIADB_ROOT_PASSWORD=RootPass
MARIADB_DATABASE=moodle
MARIADB_USER=moodle
MARIADB_PASSWORD=MoodlePass

MOODLE_ADMIN_USER=admin
MOODLE_ADMIN_PASS=Admin1234!
MOODLE_ADMIN_EMAIL=admin@example.com
```